


```
git push origin my-feature
```

- (4.) Edit, stage, commit, and push changes on your branch as usual.

Critical: Pull the latest changes regularly from the remote repository to keep your local repository up to date.

- (5.) Once your branch is ready to be merged, create a pull request on GitHub:

- Navigate to the repository on GitHub, click “Compare & pull request”.
- Respond to comments and update the pull request as needed.
- Once approved, the pull request will be merged into the development or the main branch.

>> **Linking repositories with submodules**

- (1.) Use a submodule when part of one repository (private content, shared utilities, a vendored dependency) needs its own version history but should appear as a subdirectory of the parent. The parent pins a specific commit of the submodule, so updates are explicit and reproducible.

- (2.) Add a submodule to the parent repository.

```
git submodule add gh-other-repo-url.git submodule/path/in/parent
git commit -m "Add submodule/path/in/parent as submodule"
```

- (3.) When others clone the parent, initialize submodules in the same step or after the fact:

```
git clone --recurse-submodules gh-parent-repo-url.git
git submodule update --init --recursive
```

- (4.) To update the submodule to a newer commit, update the pointer in the parent.

```
cd submodule/path/in/parent
git pull origin main
cd ..
git add submodule/path/in/parent
git commit -m "Bump submodule pointer"
```

Critical: Editing files inside the submodule does *not* automatically update the parent's pointer. You must commit twice: once inside the submodule, once in the parent.

- (5.) *Optional:* To remove a submodule:

```
git rm path/in/parent
git commit -m "Remove submodule path/in/parent"
```

>> **Preserving file history when renaming or splitting repositories**

- (1.) For renames *within* a single repository, use `git mv` rather than a plain shell `mv` followed by `git add`.

- (2.) Confirm Git detected the rename.

```
git status          # "renamed: old -> new"
git log --follow new # history across the rename
```

- (3.) To move files *between* repositories while preserving history, `git mv` is not enough — it only operates within one repository. Use `git filter-repo` to extract the relevant history into a portable form, then merge it into the target repository.

T > **Repository transfer**

(1.) Audit the repository.

- No pending branches, unmerged pull requests, or unpublished changes?
- No dependencies on private data or credentials?

(2.) Finalize README.md to reflect the latest project title, description, and status of the project. 

(3.) Create a final release tag so there is a citable, frozen snapshot of the code at time of handoff.

```
git tag v1.0.0-final
git push origin v1.0.0-final
```

(4.) *Optional:* For published work, link the repository to Zenodo. 

(5.) *Critical:* If the repository is associated with a personal GitHub account, transfer the repository to a designated maintainer, the lab's GitHub account or the institutional GitHub organization. Ensure a maintainer is assigned for substantial software projects. 

(6.) Remove the departing member's write/admin access after transfer is complete.

(7.) Archive the repository if it is no longer maintained. Archived repositories are read-only.

(8.) Add the repository title, archive status, and DOI to the lab's knowledgebase or data inventory.

 Notes (available online)

